

HTTP Protocol

IF3110 – Web-based Application Development
School of Electrical Engineering and Informatics
Institut Teknologi Bandung

Reference

- Leon Shklar and Richard Rosen, **Web application architecture: principles, protocols, and practices**, John Wiley & Sons Ltd (2003)
- <https://httpwg.org/specs/>

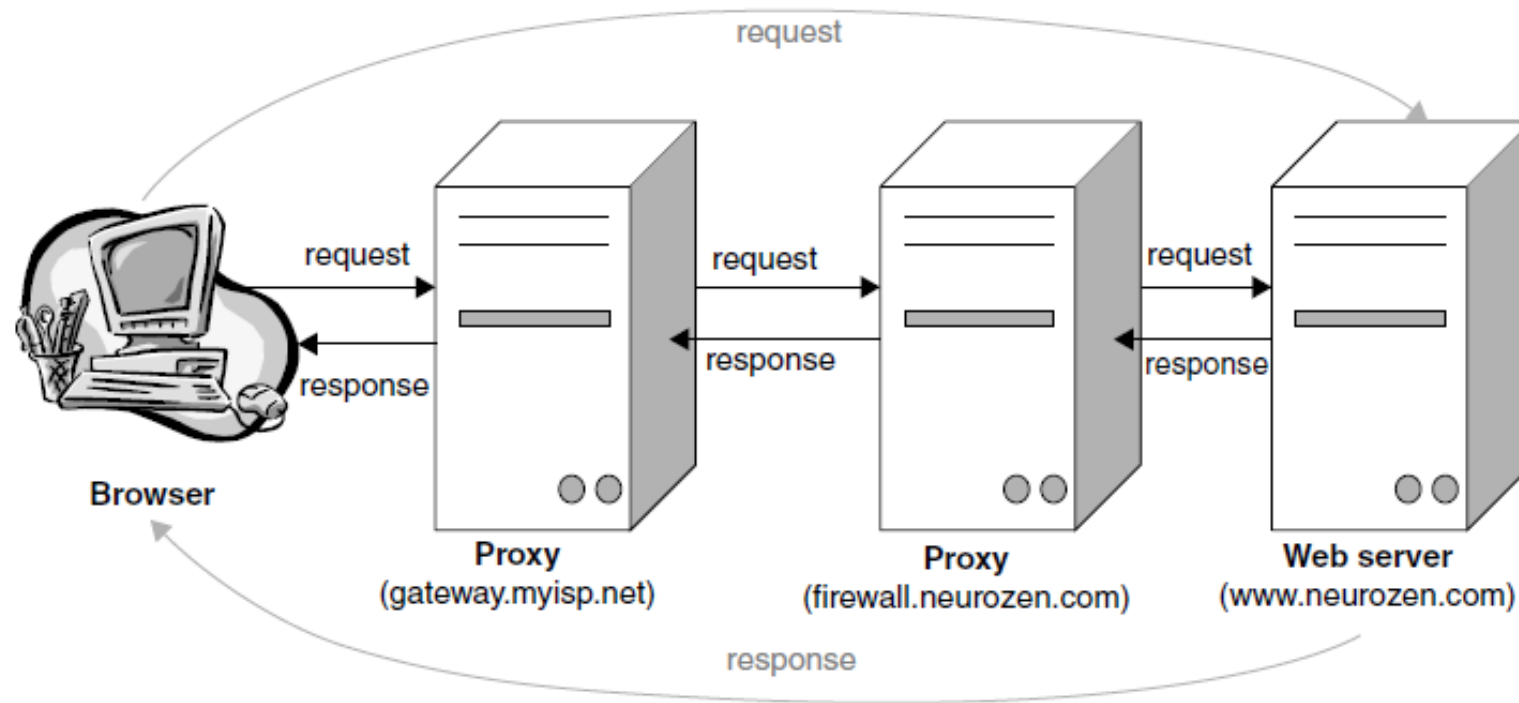
About HTTP

- Basic Ingredient Protocol for World Wide Web
- Simple – strength and weakness
- Doesn't manage the state with limited functionality
- Application Layer Protocol founded by TCP

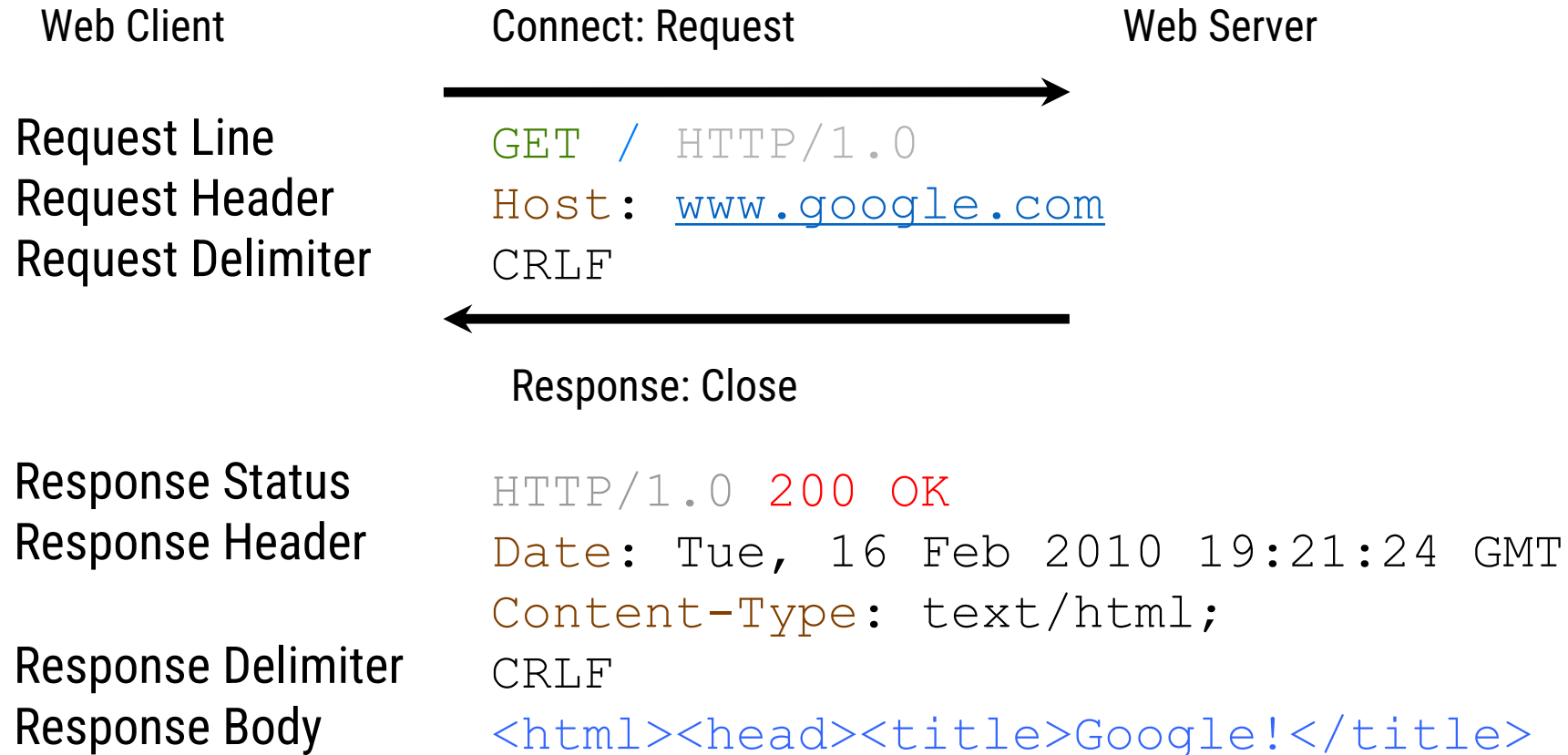
HTTP Protocol and Structure

- *request* and *response* paradigm
- *request* and *response* has similar structure
 - couple lines of *header*
 - empty line, followed *message body*
- *Stateless protocol* – a transaction is composed of a single *request* from *client* and a *response* from *server*

Request Response Virtual Circuit



Anatomy of HTTP 1.0



HTTP Request & Response Structure

```
METHOD /path-to-resource HTTP/version-number  
Header-Name-1: value  
Header-Name-2: value  
  
[ optional request body ]
```

Request

```
HTTP/version-number status-code message  
Header-Name-1: value  
Header-Name-2: value  
  
[ response body ]
```

Response

- *Request Methods:* **GET, POST, HEAD, PUT, DELETE, TRACE, OPTION, CONNECT, PATCH, QUERY (new)**

HTTP Header

- General Header

- `Date: Sun, 11 Feb 2001 22:28:31 GMT`
Date time created *message*
- `Connection: Close`
Client or Server define whether the connection is maintained/not

- Request Header

- `User-Agent: Mozilla/4.75 [en] (WinNT; U)`
browser user agent
- `Host: www.neurozen.com`
to support virtual host
- `Referer: http://www.cs.rutgers.edu/index.html`
URL of the referral

HTTP Header

- Response Header

- Location: `http://www.mywebsite.com/Page.html`
Page intended to visit (*redirect*)
- Server: `Apache/1.2.5`
Server ID

- EntityHeader

- Content-Type: `mime-type/mime-subtype`
type of message body
- Content-Length: `xxx`
length of message body
- Last-Modified: `Sun, 11 Feb 2001 22:28:31 GMT`
modification date of the content

HTTP Response Status Code

- Inform browser or proxy whether *response* is as expected
 - 1xx information
 - 2xx success
 - 3xx redirection
 - 4xx client request error
 - 5xx server error

HTTP Request & Response Example (HTML)

```
GET / HTTP/1.1
Host: google.com
User-Agent: curl/8.7.1
Accept: */*
```

Request

```
HTTP/1.1 301 Moved Permanently
Location: http://www.google.com/
Content-Type: text/html; charset=UTF-8
Content-Security-Policy-Report-Only: object-src 'none';base-uri 'self';script-src 'nonce-4GzXySxY
3K4nLDM01Pws3Q' 'strict-dynamic' 'report-sample' 'unsafe-eval' 'unsafe-inline' https: http;;repor
t-uri https://csp.withgoogle.com/csp/gws/other-hp
Date: Sun, 30 Aug 2026 07:03:35 GMT
Expires: Tue, 29 Sep 2026 07:03:35 GMT
Cache-Control: public, max-age=2592000
Server: gws
Content-Length: 219
X-XSS-Protection: 0
X-Frame-Options: SAMEORIGIN

<HTML><HEAD><meta http-equiv="content-type" content="text/html; charset=utf-8">
<TITLE>301 Moved</TITLE></HEAD><BODY>
<H1>301 Moved</H1>
The document has moved
<A HREF="http://www.google.com/">here</A>.
</BODY></HTML>
```

Response

HTTP Request & Response Example (JSON)

The screenshot shows the Chrome DevTools Network tab. The 'Headers' panel is expanded, displaying the following details for the selected request:

- General:**
 - Request URL: `https://api-edunex.cognisia.id/login/me`
 - Request Method: `GET`
 - Status Code: `200 OK`
 - Remote Address: `20.212.115.238:443`
 - Referrer Policy: `strict-origin-when-cross-origin`
- Response headers:**
 - Access-Control-Allow-Origin: Select an element in the page to inspect it - * C
 - Cache-Control: `no-cache, private`
 - Connection: `keep-alive`
 - Content-Encoding: `gzip`
 - Content-Type: `application/json`
 - Date: `Sun, 30 Aug 2026 09:47:40 GMT`
 - Server: `nginx`
 - Transfer-Encoding: `chunked`
 - Vary: `Accept-Encoding`
 - Vary: `Origin`
 - Via: `kong/2.4.0`
 - X-Kong-Proxy-Latency: `0`

The bottom status bar indicates: 450 requests | 13.6 MB transferred | 10.4 MB resources | Finish: 7.05 s | D X-Kong-Proxy-Latency

The screenshot shows the Chrome DevTools Network tab with the 'Response' panel expanded, displaying the JSON response for the selected request:

```
1 {
  "id": 247801,
  "username": " ",
  "name": " ",
  "password_status": " ",
  "email": " ",
  "is_active": 1,
  "level": "STUDENT",
  "lptk_id": 2,
  "deleted_at": null,
  "remember_token": null,
  "created_at": "2026-08-11 09:13:16",
  "updated_at": "2026-08-22 08:33:10",
  "created_by": null,
  "updated_by": null,
  "deleted_by": null,
  "thumbnail": "https://lh3.googleusercontent.com/Va/ACg8ocKHThw7gv0UxJ3oqAqQZ5H6D",
  "last_online": "2026-08-30 16:46:06",
  "faculty_id": null,
  "major_id": 1808,
  "notification_email": null,
  "zoom_email": null,
  "period_id": 26,
  "code": " ",
  "user_id": 247801,
  "place_of_birth": "-",
  "date_of_birth": "2020-01-01",
  "password": " "
}
```

The bottom status bar indicates: 454 requests | 13.6 MB transferred | 10.4 MB resources | Finish: 1 min | D Line 1, Column 113

HTTP Request Methods

Method	Safe	Idempotent	Cacheable
<u>GET</u>	Yes	Yes	Yes
<u>HEAD</u>	Yes	Yes	Yes
<u>OPTIONS</u>	Yes	Yes	No
<u>TRACE</u>	Yes	Yes	No
<u>PUT</u>	No	Yes	No
<u>DELETE</u>	No	Yes	No
<u>POST</u>	No	No	Conditional*
<u>PATCH</u>	No	No	Conditional*
<u>CONNECT</u>	No	No	No
<u>QUERY</u>	Yes	Yes	Yes

* POST and PATCH are cacheable when responses explicitly include [freshness](#) information and a matching [Content-Location](#) header.

Source:
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Methods>

GET vs POST

GET

- can be cached
- remain in the browser history
- can be bookmarked
- Data is visible to everyone in the URL
- have length restrictions
- should be used only to retrieve data
- Only ASCII characters allowed

POST

- Default : never be cached
- do not remain in the browser history
- cannot be bookmarked
- Data is not displayed in the URL
- have no restrictions on data length
- No restrictions. Binary data is also allowed

Request Methods

- HEAD
 - Similar to GET, but server **MUST NOT** return a message body in the response
 - Server only returns *header*
 - To support *cache* with *content modification information* (*Last-Modified*)

Request Methods

- PUT
 - to store a resource on a particular URI
 - if the URI refers to existing resource then the resource is being updated
- DELETE
 - to delete a resource.
- TRACE
 - send back the request received by the server
 - client can identify what the additional info added in a HTTP request (e.g., by http proxy)

Request Methods

- OPTIONS
 - return HTTP methods supported by the server on a particular URL
- CONNECT
 - convert request into a transparent TCP/IP tunnel,
 - used in SSL-encrypted used in HTTPS
- PATCH
 - used in modification on a part of the resource
- QUERY
 - Similar to GET, but it has request body content

Request Methods

- GET
 - most simple
 - doesn't contain *request body*
 - *request parameters* will be added in the URL *query string* (after“?”)
 - Only retrieve the resource/data, without any other effects (inc. modification/deletion)
- POST
 - *request body* contains *request* parameters
 - Submits data to be processed to a specified resource
 - URL doesn't contain any data (suitable for submit FORM)

Form Processing

- Form w/ POST method, will use HTTP POST to send data, with

`content-type: application/x-www-form-urlencoded`

- Query parameter will be provided as pairs of type: value
- File upload uses

`content-type: multipart/form-data`

Form Processing (Example 1)

```
POST /enlighten/calais.asmx/Enlighten HTTP/1.1
Host: api.opencalais.com
Content-Type: application/x-www-form-urlencoded
Content-Length: length

licenseID=string&content=string&paramsXML=string
```

Form Processing (Example 2)

```
POST /upload HTTP/1.1
Host: example.com
Content-Type: multipart/form-data; boundary=----
WebKitFormBoundary7MA4YWxkTrZu0gW

-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="username"

john_doe
-----WebKitFormBoundary7MA4YWxkTrZu0gW
Content-Disposition: form-data; name="profile_picture";
filename="profile.jpg"
Content-Type: image/jpeg

[Binary data of the JPEG file]
-----WebKitFormBoundary7MA4YWxkTrZu0gW
```

Adapted from <https://medium.com/@muhebollah.diu/understanding-multipart-form-data-the-ultimate-guide-for-beginners-fd039c04553d>

Form Processing (Example 3)

```
POST /api/users HTTP/1.1  
Host: example.com  
Content-Type: application/json  
Content-Length: 53
```

```
{  
  "name": "Alex",  
  "email": "alex@example.com"  
}
```

Virtual Hosting (e.g. : Apache, Nginx, etc)

```
* Host eatingbirdfood.wordpress.com:80 was resolved.
* IPv6: (none)
* IPv4: 192.0.78.12, 192.0.78.13
* Trying 192.0.78.12:80...
* Connected to eatingbirdfood.wordpress.com (192.0.78.12) port 80
> GET / HTTP/1.1
> Host: eatingbirdfood.wordpress.com
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 301 Moved Permanently
< Server: nginx
< Date: Sun, 30 Aug 2026 07:14:46 GMT
< Content-Type: text/html
< Content-Length: 162
< Connection: keep-alive
< Location: https://eatingbirdfood.wordpress.com/
< Alt-Svc: clear
< Server-Timing: a8c-cdn, dc;desc=sin, cache;desc=BYPASS;dur=0.0
<
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>
* Connection #0 to host eatingbirdfood.wordpress.com left intact
```

```
* Host onepiecethenewworld.wordpress.com:80 was resolved.
* IPv6: (none)
* IPv4: 192.0.78.12, 192.0.78.13
* Trying 192.0.78.12:80...
* Connected to onepiecethenewworld.wordpress.com (192.0.78.12) port 80
> GET / HTTP/1.1
> Host: onepiecethenewworld.wordpress.com
> User-Agent: curl/8.7.1
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 301 Moved Permanently
< Server: nginx
< Date: Sun, 30 Aug 2026 07:15:02 GMT
< Content-Type: text/html
< Content-Length: 162
< Connection: keep-alive
< Location: https://onepiecethenewworld.wordpress.com/
< Alt-Svc: clear
< Server-Timing: a8c-cdn, dc;desc=sin, cache;desc=BYPASS;dur=0.0
<
<html>
<head><title>301 Moved Permanently</title></head>
<body>
<center><h1>301 Moved Permanently</h1></center>
<hr><center>nginx</center>
</body>
</html>
* Connection #0 to host onepiecethenewworld.wordpress.com left intact
```

*It has same IP, but with different Subdomain

Session Management

```
GET /movies/register HTTP/1.1  
Host: www.sample-movie-rental.com  
Authorization:...
```

```
HTTP/1.1 200 OK  
Set-Cookie: CLIENT=Rich; path=/movies  
...
```

```
GET /movies/rent-recommended HTTP/1.1  
Host: www.sample-movie-rental.com  
Cookie: CLIENT=Rich
```

Cookies (Example)

▼ Response headers		☐ Raw
Access-Control-Allow-Credentials	true	
Connection	keep-alive	
Content-Length	51	
Content-Security-Policy	frame-ancestors self;	
Content-Type	text/html; charset=utf-8	
Date	Sun, 30 Aug 2026 10:59:38 GMT	
Location	http://167.205.88.188/	
Referrer-Policy	strict-origin-when-cross-origin	
Server	nainx	
Set-Cookie	token=[REDACTED] [REDACTED]; Path=/; Expires=Mon, 31 Aug 2026 10:59:38 GMT; HttpOnly; Secure; SameSite=None	
Strict-Transport-Security	max-age=31536000; includeSubDomains	
Vary	Origin, Accept	
X-Content-Type-Options	nosniff	
X-Frame-Options	SAMEORIGIN	
X-Powered-By	Express	
X-Xss-Protection	1; mode=block	

▼ Request Headers		☐ Raw
Accept	*/*	
Accept-Encoding	gzip, deflate, br, zstd	
Accept-Language	en-US,en;q=0.5	
Connection	keep-alive	
Cookie	_ga=GA1.1.[REDACTED]; _ga_80C1VZL9M7=GS2.1.[REDACTED]; token=[REDACTED]	
Host	museum.itb.ac.id	
Next-Router-Prefetch	1	
Next-Router-Segment-Prefetch	/!KHB1YmxpYyk/about-us	
Next-Url	/profile	
Referer	https://museum.itb.ac.id/profile	

Basic Auth

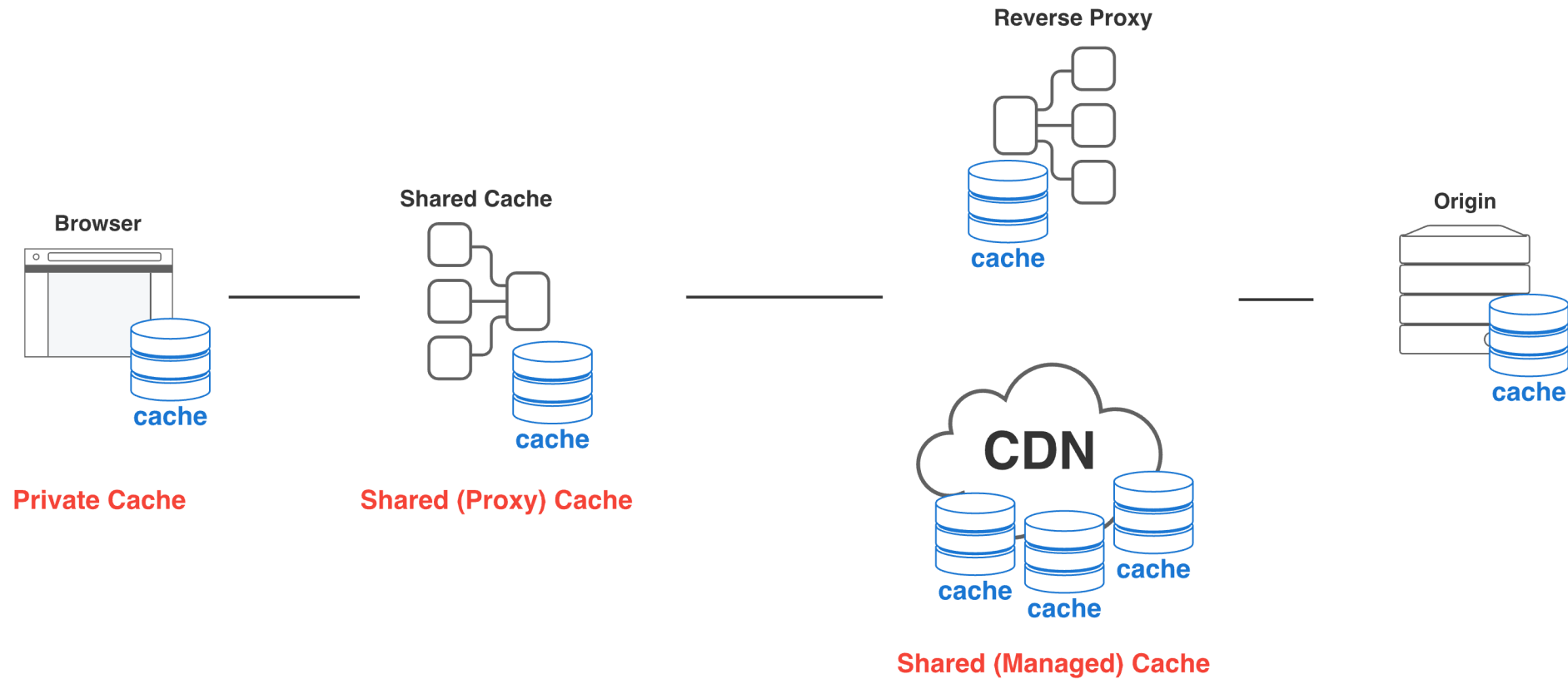
```
HTTP/1.1 401 Authenticate  
Date: Mon, 05 Feb 2001 03:41:23 GMT  
Server: Apache/1.2.5  
WWW-Authenticate: Basic realm="Chapter3"
```

```
GET /book/chapter3/index.html HTTP/1.1  
Date: Mon, 05 Feb 2001 03:41:24 GMT  
Host: www.neurozen.com  
Authorization: Basic eNCoDEd-uSErId:pASswORD
```

Bearer Auth

▼ Request Headers ☐ Raw	
Accept	application/json, text/plain, */*
Accept-Encoding	gzip, deflate, br, zstd
Accept-Language	en-US,en;q=0.9
Authorization	Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9.eyJhdWQiOiI0IiwianRpIjoiMTBmODhmYjMxMmMwLUEyZDhmE4YyZiNmE4M2E2YyM2Y2Nh7DYwN2Vl7DM4uNTkuN2Q4MjQyMiRlO gjm99VWJzGleDwtN0EoJAAETA0aQJEw2RtDLnex78yopGeyC257MC4DzSNExy52S 9z63WuUKSkim3nSshwxd2xR14_sYcZv_YkiQBegzNP5aGznQf92694UOH_I88cTCG H-KOElYt0Xrli4zyL6uewvZSFcG6GYvNYsXI7azEQcw6MsCbFIResifkUmv3A
Connection	keep-alive
Host	api-edunex.cognisia.id
Origin	https://edunex.itb.ac.id
Referer	https://edunex.itb.ac.id/
Sec-Ch-Ua	"Not=A?Brand";v="99", "Google Chrome";v="151", "Chromium";v="151"
Sec-Ch-Ua-Mobile	?1
Sec-Ch-Ua-Platform	"Android"
Sec-Fetch-Dest	empty
Sec-Fetch-Mode	cors
Sec-Fetch-Site	cross-site
User-Agent	Mozilla/5.0 (Linux; Android 15; Pixel 9) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/151.0.0.0 Mobile Safari/537.36

HTTP Caching



Source: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Caching>

Browser Caching

- The browser inspects the headers of the HTTP response generated by the web server. There are four headers commonly used for caching:
 - Cache-Control
 - ETag
 - Expires
 - Last-Modified
- Browser has Cache API for storing and retrieving network requests and their corresponding responses

Name	Status	Type	Size
version.json?t=1788085324226	200	xhr	2.1 kB
Pasti-RYf7oCwD.svg	200	svg+xml	(disk cache)
news?limit=1000&page=1	304	xhr	0.5 kB
data:image/png;base...	200	png	(memory cache)
data:image/png;base...	200	png	(memory cache)
KFO7CnqEu92Fr1ME7kSn66aGLd...	200	font	(memory cache)
logo_48.png	200	png	(memory cache)
recaptcha__en.js	200	text/javascript	(disk cache)
js?l=dataLayer&id=G-NTY8JYPN5M	200	script	(memory cache)

Application						
https://pasti.kemenkum.go.id						
Bucket name Default bucket						
#	Name	Respons...	Content-...	Content-...	Time Ca...	Vary Hea...
0	/assets/BSKH-BptmJSnE.js	basic	applicati...	52	8/24/202...	
1	/assets/BannerCard.vue_vue_type_script_setu...	basic	applicati...	545	8/30/202...	
2	/assets/BlankLayout-D0Ksc7Zk.js	basic	applicati...	230	8/30/202...	
3	/assets/Blog-D2FbLwjQ.js	basic	applicati...	2,243	8/30/202...	
4	/assets/Dashboard-B1pKBdwu.js	basic	applicati...	983	8/30/202...	
5	/assets/DefaultDashboard-B-cOCITT.js	basic	applicati...	15,535	8/30/202...	
6	/assets/Disabled-D5mB3LDv.js	basic	applicati...	263	8/30/202...	
7	/assets/Edit-tiOuqCKT.js	basic	applicati...	5,359	8/30/202...	

Cache-Control

▼ Response Headers

```
accept-ranges: bytes
age: 316770
alt-svc: quic=":443"; ma=2592000; v="39,38,37,36,35"
cache-control: public, max-age=31536000
content-encoding: gzip
content-length: 10718
content-type: text/javascript; charset=UTF-8
date: Thu, 03 Aug 2017 19:22:33 GMT
expires: Fri, 03 Aug 2018 19:22:33 GMT
last-modified: Thu, 03 Aug 2017 15:18:22 GMT
server: sffe
status: 200
vary: Accept-Encoding, Origin
x-content-type-options: nosniff
x-xss-protection: 1; mode=block
```

Source:

<https://www.imperva.com/learn/performance/cache-control/>

Cache-control is an HTTP header used to specify caching policies in both client requests and shared cache (CDN, Proxy). Some directives:

Directive	Description
public	indicates that a resource can be cached by any cache
private	resource is user specific—it can still be cached, but only on a client device
no-cache	indicates that the response can be stored in caches , but the response must be validated with the origin server before each reuse
no-store	indicates that any caches of any kind (private or shared) should not store this response.
max-age=N	indicates that the response remains fresh until N seconds after the response is generated.

Other Cache Related Header

- **Etag** response header is an identifier for a specific version of a resource
- **Expires** response header contains the date/time after which the response is considered expired
- **Last Modified** response header contains a date and time when the origin server believes the resource was last modified.

×	Headers	Preview	Response	Initiator	Timing	Adblock
	Content-Type		text/html			
	Date		Sun, 30 Aug 2026 10:17:15 GMT			
	Etag		"a8e51e87b9fe7dc09becf56f67707250"			
	Expires		Sun, 30 Aug 2026 02:31:39 GMT			
	Last-Modified		Sun, 30 Aug 2026 01:15:36 GMT			
	Referrer-Policy		strict-origin-when-cross-origin			
	Server		Google Frontend			
	Strict-Transport-Security		max-age=63072000			
	Vary		Accept-Encoding			

Persistent Connection

- HTTP 1.0 uses TCP separately for each request
 - not efficient
 - slow (i.e., high-latency)
- HTTP 1.1 uses a persistent connection, can be used by several requests

`Connection: Close`

`Connection: Keep-Alive`

HTTP 1.1 vs 1.0

- Additional Methods (PUT, DELETE, TRACE, CONNECT + GET, HEAD, POST)
- Additional Headers
- Transfer Coding (chunk encoding)
- Persistent Connections (content-length matters)
- Request Pipelining

HTTP Sometime Back

- Using HTTP 1.1 since 1997 / 1999
 - Connection: keep-alive
 - Head of Line Blocking
- But we still use N TCP Connections per origin
- No Header Compression
- And Many Hacks because requests are evil
 - Spriting of Images
 - Resource Inlining
 - Concatenation of files
 - Domain Sharding
 - CDNs

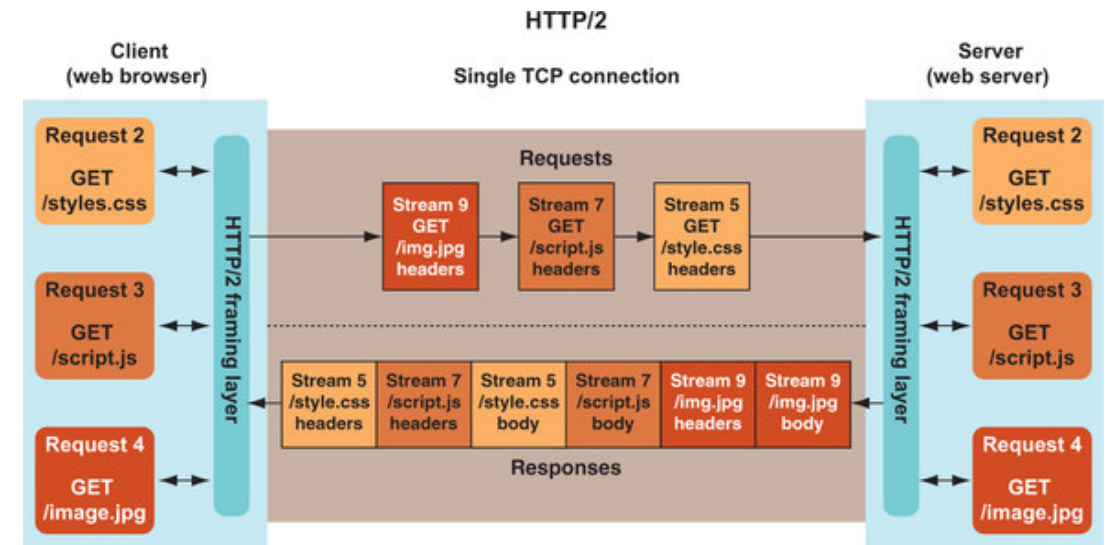
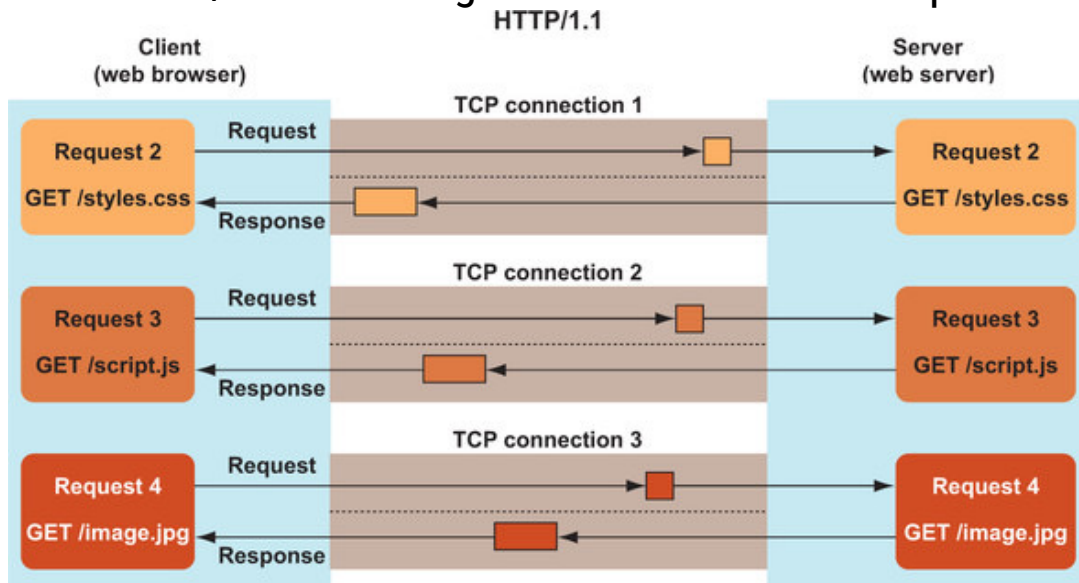
HTTP/2

Addressing HTTP 1.1 Performance Issues:

- Binary rather than textual protocol
- Multiplexed rather than synchronous
- Flow control
- Stream prioritization
- Header compression
- Server push

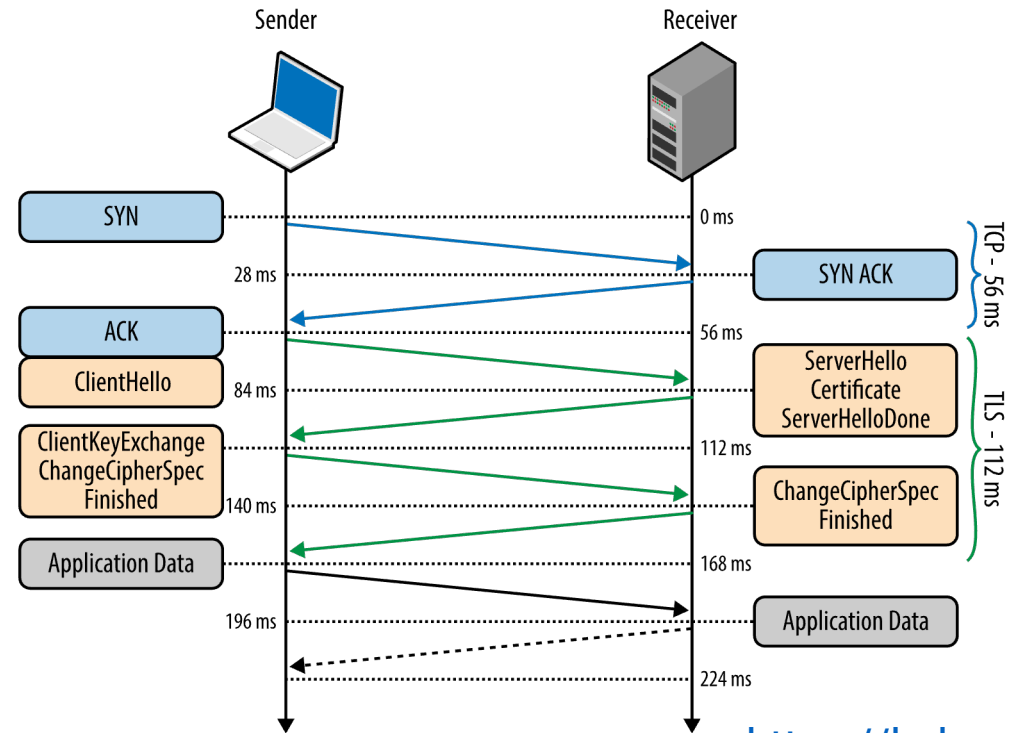
Multiple TCP connection vs Single Connection

- Multiple HTTP/1 requests in parallel require multiple TCP connections.
- Most browsers open up to six connections per domain in parallel for this reason.
- But when the max. connection reached => blocking occurs. Also, TCP connections are expensive.
- A technique called domain sharding splits resources into many domains to address this limitation.
- HTTP/1 may allow "persistent connection": using `connection:keep-alive` header to use one connection for several requests. However this introduce *head-of-line blocking*: the current request must complete before the next one can be sent.
- HTTP/2 allows single TCP connection multiplex streams of resources over frames which may be interleaved.



HTTPS

- Is HTTP over TLS over TCP
- Hence, TCP 3 way handshake and certificate exchange is performed before actual HTTP protocol application data.



<https://hpbn.co/>

Why HTTP/2

- Performance matters
 - HTTP/2 reduces the impact of latency on web applications
- TLS is becoming the default
 - HTTP/2 amortizes TLS costs for the entire application
- Enabling new web development
- User perceivable improvement in web site performance
- Work with today's internet
- Remain compatible with existing content

What is HTTP/2

- Used SPDY3 as its first draft
- Main Driven by Performance
- But also includes
 - Security
 - Reliability

HTTP 1.x vs HTTP/2

HTTP 1.x

Text protocol

Verb & Resource Semantics

Headers and body

Headers repeated

SSL Optional

Single connection – single request

HTTP 2

Binary protocol

Verb & Resource Semantics

Headers and body

Headers optimized

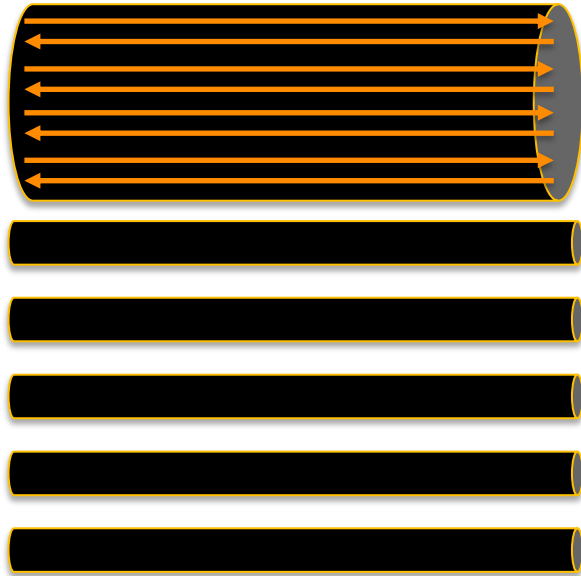
SSL optional (mostly used)

Single connection – multiple request

Compression

HTTP/2 Connections and Streams

HTTP/1.1 – Request = Connection



HTTP/2 – Request = Stream



Each request required dedicated TCP connection	TCP connection can have multiple streams (requests)
Responses come in order per connection	Responses can come out of order, server can optimize
Each connection requires setup + slow start	No connection setup for new streams, no slow start
Application sees “connections”	Streams are represented as “connections” to apps

Header Compression

▶ HPACK

Static Table

:path: /
accept-encoding:
gzip,deflate
user-agent:
:authority:
...

Dynamic
Table

:authority:
www.example.com

:method: GET
:scheme: http
:path: /
:authority:
www.example.com



Header Compression

- **Each header is expressed as either...**
 - **Indexed:** Reference to full header in static or dynamic table
 - **Literal:**
 - Name as reference to static/dynamic table entry, or as Huffman-encoded string
 - Value as Huffman-encoded string
 - Indexing behavior:
 - Add to dynamic table
 - Don't add to dynamic table
 - NEVER add to dynamic table (supposed to be persisted through intermediaries)

Request Reliability

- In HTTP no retry a request when an error occurs
 - Re-Attempt: It is only possible to some server to do some processing prior to the error which could result in undesired effects
- HTTP/2 Provides 2 mechanisms as guarantee to a client that a request has not been processing
 - GOAWAY: the highest stream number that has been processed
 - REFUSED_STREAM: error code can be included in a RST_STREAM frame

HTTP/2 vs HTTP/3

HTTP/2

TCP + TLS

Multiplexing at the HTTP layer

A lost TCP packet can stall all streams

Header compression: HPACK

Connection tied to an IP address and port

HTTP/3

QUIC over UDP; TLS 1.3 integrated

Streams managed by the transport layer

Loss mainly stalls the affected stream

Field compression: QPACK

Connection IDs enable path migration

HTTP/3 runs over QUIC

HTTP semantics remain unchanged: methods, status, fields, and body

QUIC provides stream multiplexing and per-stream flow control

The QUIC handshake combines transport parameters with TLS 1.3

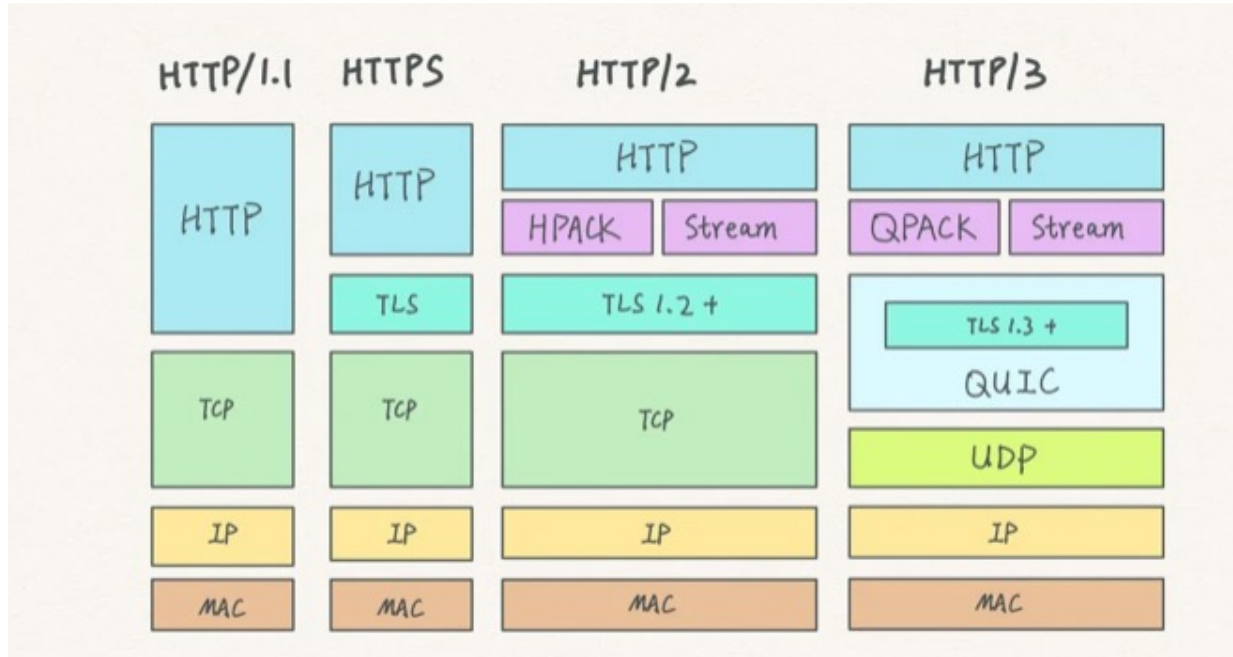
QUIC packets are carried in UDP datagrams

ALPN uses the h3 token; endpoints can be advertised with Alt-Svc

Why HTTP/3?

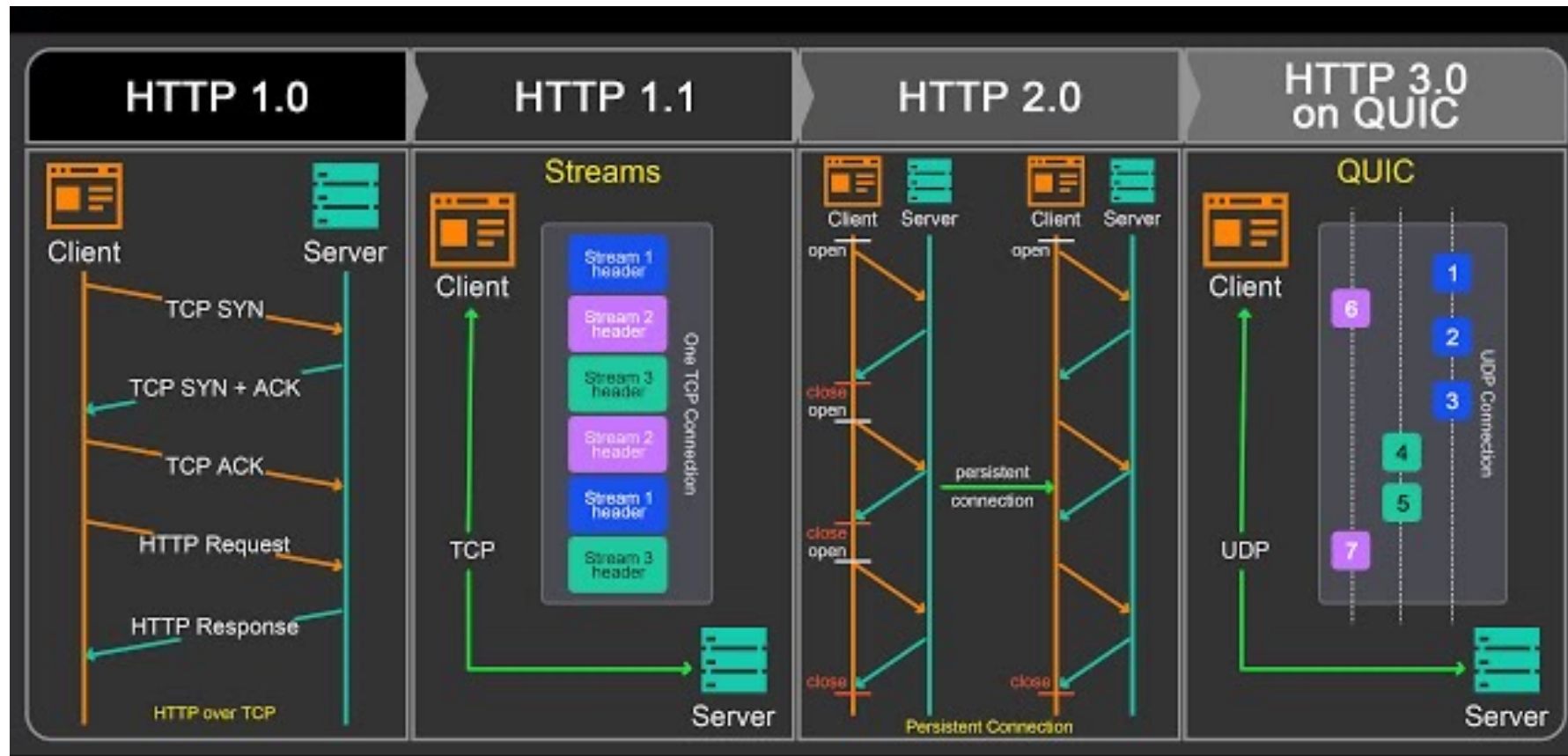
- HTTP/2 carries many streams over one TCP byte stream
 - TCP must deliver bytes in order
- One lost packet can delay transactions across an HTTP/2 connection
 - QUIC recovers data per stream; unrelated streams can continue
 - Congestion control still operates at the connection level

QPACK compresses HTTP/3 fields



<https://www.rumahweb.com/journal/http3-adalah/>

- QPACK is designed for QUIC streams that may arrive out of order
- A static table stores common fields; a dynamic table grows during the connection
- Encoder and decoder streams carry compression instructions separately
- Unreceived dynamic-table references can block an individual stream
- The encoder balances compression efficiency against blocking risk



<https://www.youtube.com/watch?v=UMwQjFzTQXw>

Faster handshakes and migration

- The QUIC handshake combines cryptographic and transport negotiation
 - New connections can send application data sooner than a TCP + TLS sequence
- Previously used connections may send 0-RTT data
 - 0-RTT data can be replayed: use it only for replay-safe operations
- Connection IDs preserve a connection when an IP address or port changes

HTTP/3 adoption still needs fallback

- Advertise h3 support through Alt-Svc or another suitable discovery mechanism
- Prepare UDP, TLS 1.3, load balancers, observability, and rate limiting
- Some networks block or restrict UDP
- If QUIC fails, clients should try a TCP-based HTTP version
- Keep HTTP/2 or HTTP/1.1 available and measure results on real networks